Color Sorting and Stacking

1. Game Introduction

- Start Docker

After entering the main board desktop, open the terminal and run the following command to start the Dofbot corresponding container:

```
./Docker_Ros.sh
```

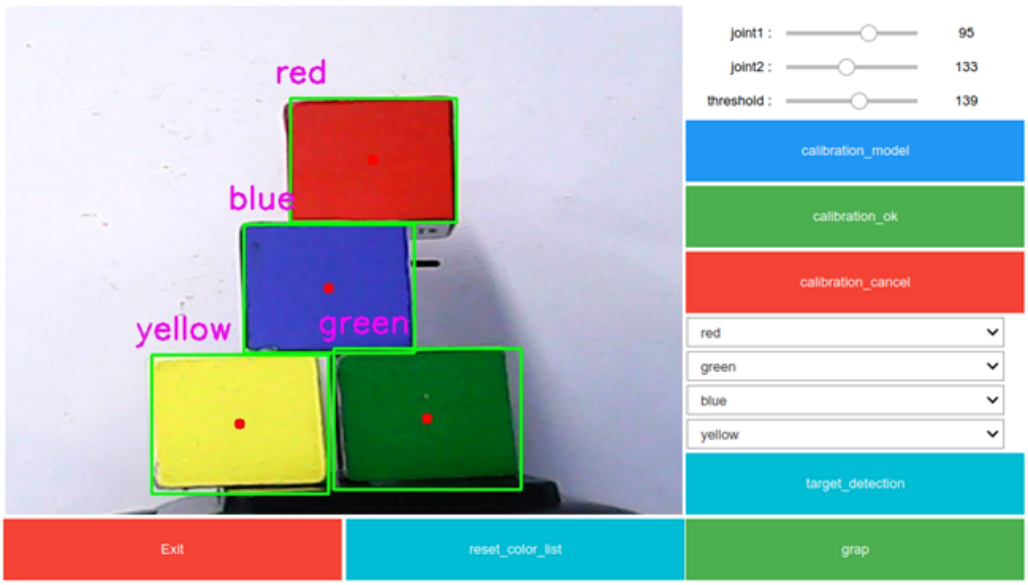
Then enter the following command in the docker terminal to start jupyter-lab,

```
cd
jupyter-lab --allow-root
```

Then enter the main board's IP address plus :9999 in the browser,

```
IP:9999         // Example: 192.168.1.11:9999
```

```
Game instructions:

1) First, run without the main program running.

2) Enter the docker container and start a command line window as the inverse
solution service, running continuously.

ros2 run dofbot_info kinemarics_dofbot

3) Color sorting game path:
/root/Dofbot_2D/src/dofbot_color_identify/scripts/Color_sorting.ipynb

4) Color stacking game path:
/root/Dofbot_2D/src/dofbot_color_stacking/scripts/Color_stacking.ipynb
```



```
(1) HSV calibration: Due to different environments, color recognition produces
different results. So before playing color sorting, it is best to perform HSV
calibration. For specific methods, refer to [Color Calibration].

(2) Frame calibration: After color calibration is completed, frame calibration is
required to obtain the position information of blocks within the frame. For
specific calibration methods, refer to [Visual Positioning].

(3) Color selection: Players can freely select the color to grab according to
their needs, and the robotic arm will sort according to the color selected by the
player.

(4) Grab and sort: After the camera recognizes the blocks, click the grab
button.

(5) Clear colors: If players want to change the selected color, clear the
selected colors and re-select for recognition.

(6) Exit program: To close this game, please click the exit button to release
resources.
```

2. Color Recognition

Note: Frame calibration must be completed to be effective.

- Operation Process

  (1) After calibration is completed, select colors in the color selection box on the right side (no color selected by default), as shown in the figure above.

  (2) After color selection is completed, click the [target_detection] button to call the color recognition function and mark the recognized blocks.

  (3) Click the [grap] button to sort out the recognized blocks.

  (4) Click the [reset_color_list] button to clear the selected color sequence. The color selection bar will still show the last selection order. Just change it and repeat steps (2), (3).

- Code Design

Color selection sequence, generate color sequence after selection is completed

```
def color_list_one_Callback(value):
    pass

def color_list_two_Callback(value):
    pass

def color_list_three_Callback(value):
    pass

def color_list_four_Callback(value):
    pass

color_list_one.observe(color_list_one_Callback)
color_list_two.observe(color_list_two_Callback)
color_list_three.observe(color_list_three_Callback)
color_list_four.observe(color_list_four_Callback)
```

Click the [target_detection] button to call the color recognition function select_color()

```
def select_color(self, image, color_hsv, color_list):
    '''
    Select recognition color
    :param image: Input image
    :param color_hsv: Example {color_name:((lowerb),(upperb))}
    :param color_list: Color sequence: ['0': None '1': Red '2': Green '3': Blue
'4': Yellow]
    :return: Output processed image, (color, position)
    '''
    pass
```

Parse the passed color sequence and color information, and detect blocks in the color sequence separately. Return block coordinates

```
def get_Sqaure(self,   color_hsv):
'''
Color recognition, get block coordinates
'''
pass
```

For detailed code introduction, see /root/Dofbot_2D/src/dofbot_color_identify/scripts/Color_sorting.ipynb

3. Get Inverse Solution Result

- Operation Process

  After detecting blocks, click the [grap] button to grab

- Code Design

  Send the obtained block coordinate information through ROS server communication, solve the rotation angle that each joint should rotate through inverse kinematics. Get joint angles and drive the robotic arm to grab.

```
def server_joint(self, posxy):
    '''
    Post position request, get joint rotation angle
    Publish position request, get joint rotation angle
    :param posxy: Location point x,y coordinates
    :return: Rotation angle of each joint
    '''
    # Wait for the server to start
    # 等待server端启动
    self.client.wait_for_service()
    # Create a message pack
    # 创建消息包
    request = Kinemarics.Request()
    request.tar_x = posxy[1] - 0.03
    request.tar_y = posxy[0] - 0.01
    request.tar_z = 0.06
    request.kin_name = "ik"
    request.pitch = 1.04
    print("request: ",request)
    self.get_logger().info(f"request:{request}")
```

```
    try:
        future = self.client.call_async(request)
        rclpy.spin_until_future_complete(self, future, timeout_sec=5.0)
        response = future.result()
        self.get_logger().info(f"response:{response}")
        joints = [0.0, 0.0, 0.0, 0.0, 0.0]
        joints[0] = response.joint1
        joints[1] = 180 - response.joint2
        joints[2] = 180 - response.joint3
        joints[3] = 180 - response.joint4
        joints[4] = 180 - response.joint5
        # When the inverse solution is out of bounds and a negative value
appears, adjust it appropriately.
        # 当逆解越界,出现负值时,适当调节.
        if joints[2] < 0:
            joints[1] += joints[2] * 3 / 5
            joints[3] += joints[2] * 3 / 5
            joints[2] = 0
        self.get_logger().info(f"joints:{joints}")
        return joints
    except Exception:
        print("Error!")
```

For detailed code, see identify_target.py and identify_grap.py in the /root/Dofbot_2D/src/dofbot_color_identify/scripts path

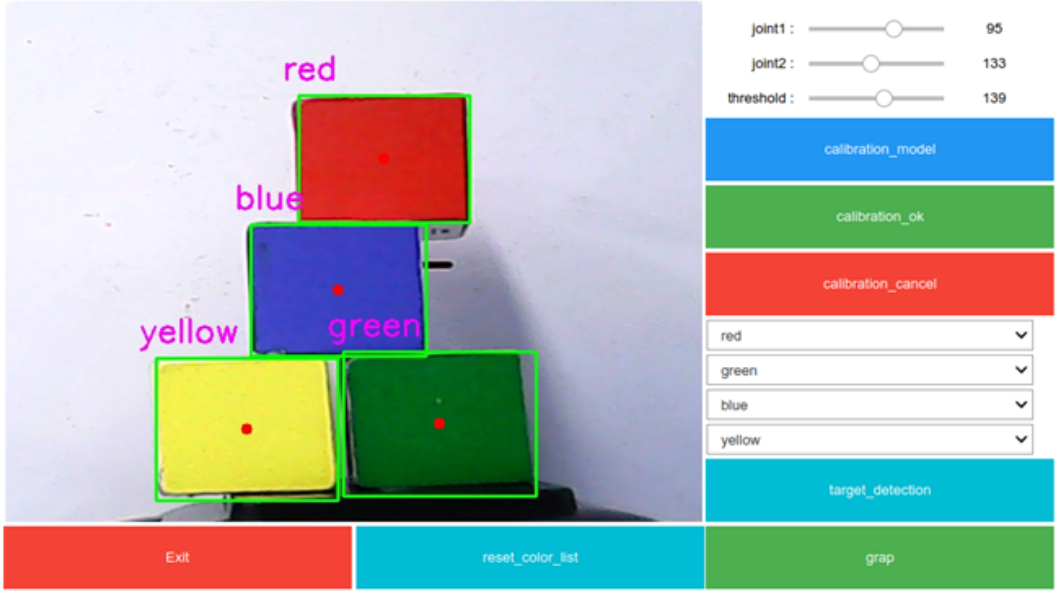
- Grab Design Process

Lift robotic arm -> Release gripper -> Move to block position -> Tighten gripper -> Lift up -> Move to target position -> Release gripper -> Robotic arm reset.

4. Color Stacking

```
The color stacking game is similar to color sorting, with the difference that the
robotic arm finally stacks the blocks vertically according to the selected
colors.

Note: Frame calibration must be completed to be effective. Color stacking not
only requires the ability to grip blocks, but the gripping position should be as
consistent as possible, with high requirements for positioning. Users can adjust
the gripping position and posture according to the actual situation. For specific
adjustment methods, please refer to the [Visual Positioning] section.
```

- Example Image



- Operation Process

  (1) After calibration is completed, select colors in the color selection box on the right side (no color selected by default), as shown in the figure above.

  (2) After color selection is completed, click the [target_detection] button to call the color recognition function and mark the recognized blocks.

  (3) Click the [grap] button to sort out the recognized blocks, then stack them in the gray frame. The stacking order is consistent with the color selection order shown above.

  (4) Click the [reset_color_list] button to clear the selected color sequence. The color selection bar will still show the last selection order. Just change it and repeat steps (2), (3). For detailed code introduction, see /root/Dofbot_2D/src/dofbot_color_stacking/scripts/Color_stacking.ipynb